

# Sistema Inteligente de Monitoramento e Cuidados de Plantas

Gaisani Calle

Dimas Vianna

2026

## Resumo

Este trabalho apresenta o desenvolvimento do Lumees Yappplant, um ecossistema inteligente baseado em Internet das Coisas (IoT), computação em nuvem e inteligência artificial dedicado ao monitoramento residencial e classificação preditiva da saúde de plantas. O sistema aborda a dificuldade na manutenção botânica doméstica decorrente da invisibilidade de variáveis microclimáticas e edáficas. Para a captura de dados, implementou-se um módulo de hardware sob o microcontrolador ESP32 integrado a sensores de temperatura, umidade do ar (DHT11), luminosidade (BH1750) e umidade de solo (higrômetro analógico). Os dados de telemetria são normalizados e transmitidos via requisições HTTP POST para uma API assíncrona desenvolvida em FastAPI. O backend consome um modelo de aprendizado supervisionado baseado no algoritmo K-Nearest Neighbors (KNN), treinado com dados sintéticos estruturados e exportado em formato binário (.pkl), para classificar o estado de saúde vegetal e vincular diagnósticos explicativos automatizados. A persistência dos históricos e estados correntes ocorre em uma base NoSQL do Firebase Firestore. Uma aplicação móvel desenvolvida em Flutter permite a visualização gráfica temporal das últimas 5 horas, exportação de relatórios em formato CSV e a renderização dinâmica de um avatar gamificado responsivo ao diagnóstico da inteligência artificial. O sistema demonstrou alta acurácia na classificação analítica e viabilidade técnica para a sincronização síncrona de dados entre hardware e interfaces móveis.

**Palavras-chaves:** Internet das Coisas (IoT). Aprendizado de Máquina. FastAPI. Flutter. Monitoramento Ambiental.

# Sumário

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Sumário</b>                                       | <b>2</b>  |
| <b>Introdução</b>                                    | <b>3</b>  |
| <b>1 Problema Identificado</b>                       | <b>3</b>  |
| <b>2 Objetivos</b>                                   | <b>3</b>  |
| 2.1 Objetivo Geral                                   | 3         |
| 2.2 Objetivos Específicos                            | 3         |
| <b>3 Metodologia</b>                                 | <b>4</b>  |
| <b>4 Arquitetura do Sistema</b>                      | <b>4</b>  |
| 4.1 Arquitetura Geral                                | 4         |
| 4.2 Diagrama de Dados                                | 5         |
| 4.3 Fluxo de Dados                                   | 6         |
| <b>5 Desenvolvimento do Hardware e IoT</b>           | <b>7</b>  |
| 5.1 Componentes Utilizados                           | 7         |
| 5.2 Esquema Elétrico e Montagem do Circuito          | 8         |
| 5.3 Integração por API                               | 8         |
| <b>6 Inteligência Artificial (Modelo KNN)</b>        | <b>9</b>  |
| 6.1 O Algoritmo KNN                                  | 9         |
| 6.2 Geração de Dados Sintéticos                      | 9         |
| 6.3 Variáveis Ambientais Consideradas                | 9         |
| 6.4 Treinamento e Avaliação do Modelo                | 10        |
| 6.5 Estrutura Condicional de Status                  | 10        |
| 6.6 Fluxo de Dados                                   | 10        |
| <b>7 Engenharia de Software e Backend</b>            | <b>11</b> |
| 7.1 Estruturação com FastAPI                         | 11        |
| 7.2 Documentação dos Endpoints (Rotas da API)        | 11        |
| 7.3 Integração Backend-Firebase                      | 11        |
| <b>8 Interface do Usuário</b>                        | <b>12</b> |
| 8.1 Fluxo de Navegação e Usabilidade (UX/UI)         | 12        |
| 8.2 Identidade Visual e Elementos Gráficos           | 13        |
| 8.3 Detalhamento das Telas Desenvolvidas             | 13        |
| 8.3.1 Telas de Acesso (Inicial, Login e Cadastro)    | 13        |
| 8.3.2 Tela Principal (Monitoramento Real-Time)       | 14        |
| 8.3.3 Tela Dashboard (Gráficos Históricos)           | 14        |
| 8.3.4 Tela de Diagnóstico da Inteligência Artificial | 14        |
| 8.3.5 Tela de Relatório (Exportação CSV)             | 14        |
| 8.3.6 Tela de Configurações e Usuário                | 14        |
| <b>Conclusão e Trabalhos Futuros</b>                 | <b>14</b> |
| 8.4 Propostas de Melhorias Futuras                   | 15        |
| <b>REFERÊNCIAS</b>                                   | <b>15</b> |

# Introdução

O avanço da Internet das Coisas (IoT) e das tecnologias de desenvolvimento web e mobile possibilita a criação de soluções integradas voltadas para o monitoramento ambiental e automação residencial. No contexto do cuidado botânico doméstico, a manutenção de plantas enfrenta desafios relacionados à falta de acompanhamento em tempo real e à ausência de diagnósticos preditivos baseados nas condições do ecossistema.

Este projeto propõe o desenvolvimento de um ecossistema inteligente que une hardware, computação em nuvem e inteligência artificial para monitorar a saúde de plantas de forma interativa. Utilizando o conceito de gamificação inspirado em bichinhos virtuais (estilos *Tamagotchi*), o sistema converte variáveis físicas em estados visuais intuitivos de humor e bem-estar, tornando o cuidado diário uma experiência mais acessível e engajadora.

## 1 Problema Identificado

A manutenção da saúde de plantas em ambientes residenciais internos apresenta desafios práticos devido à invisibilidade de variáveis ambientais importantes. Fatores como a oscilação da umidade do solo, variações térmicas e níveis inadequados de luminosidade (Lux) frequentemente resultam em desidratação, excesso de rega ou estresse lumínico antes que os sinais físicos sejam visíveis na planta. Além disso, as necessidades hídricas e climáticas alteram-se drasticamente dependendo da estação do ano (verão/primavera e inverno/outono) e do período do dia (diurno/noturno). A ausência de um mecanismo acessível que centralize esses dados e os converta em diagnósticos preventivos dificulta o cuidado correto por parte dos usuários.

## 2 Objetivos

### 2.1 Objetivo Geral

Desenvolver um sistema integrado de IoT, aplicação mobile e Inteligência Artificial para monitoramento residencial de plantas, capaz de coletar dados ambientais em tempo real e classificar o estado de saúde da planta através de um modelo preditivo.

### 2.2 Objetivos Específicos

- Construir um módulo de hardware utilizando o microcontrolador ESP32 integrado aos sensores DHT11, BH1750 e higrômetro de solo para medição automatizada de temperatura e umidade do ar, luminosidade e umidade da terra.
- Desenvolver uma API em Python utilizando o framework FastAPI para o recebimento de dados de telemetria, processamento de inteligência artificial e persistência de dados.

- Implementar um modelo de Machine Learning baseado no algoritmo KNN (*K-Nearest Neighbors*) para classificar a saúde da planta em tempo real a partir do histórico de dados.
- Desenvolver uma interface mobile responsiva via Flutter integrada nativamente ao banco de dados Firebase Firestore para visualização dos parâmetros da planta.

### 3 Metodologia

A metodologia deste projeto está dividida em quatro fases estruturais executadas de forma ágil e paralela:

1. **Modelagem e Infraestrutura de Dados:** Criação e configuração de um banco de dados NoSQL utilizando o Firebase Firestore, definindo as coleções de usuários, plantas e histórico de leituras. Geração de uma base de dados sintéticos para simular o histórico ambiental de 30 dias necessário para o aprendizado da IA.
2. **Desenvolvimento da Inteligência Artificial e Backend:** Treinamento do modelo de classificação KNN em Python com suporte da biblioteca *scikit-learn*. Construção de uma API com FastAPI para hospedar o modelo e centralizar a comunicação entre os dispositivos e o banco de dados.
3. **Construção do Hardware e Calibragem:** Montagem do circuito físico eletrônico baseado no ESP32. Realização da calibração do sensor higrômetro de solo em ambientes secos e saturados de água para garantir a acurácia dos dados enviados por requisições HTTP POST.
4. **Desenvolvimento Frontend e Integração Geral:** Construção das telas da aplicação (Autenticação, Dashboard, Visualização da Planta e Relatórios CSV) na ferramenta FlutterFlow, conectando as variáveis do Firebase à interface para permitir atualizações visuais em tempo real baseadas no diagnóstico do modelo preditivo.

## 4 Arquitetura do Sistema

### 4.1 Arquitetura Geral

O sistema é estruturado em quatro camadas principais que trabalham de forma integrada: o Hardware (sensores), o Backend (servidor e inteligência artificial), o Banco de Dados (armazenamento) e o Frontend (aplicativo móvel). Essa divisão garante que cada parte do projeto tenha uma função específica e isolada, facilitando a manutenção e a comunicação dos dados.

1. **Camada de Captura (Hardware):** Composta pelo microcontrolador ESP32 e sensores físicos. Esta camada é responsável por medir as variáveis do ambiente

(como a umidade do solo e a temperatura) e enviar esses dados brutos para o Backend através de requisições na rede.

2. **Camada de Processamento (Backend e IA):** Desenvolvida em Python com o framework FastAPI. É o "cérebro" do sistema. O Backend recebe os dados enviados pelo Hardware, analisa e trata os dados brutos, classifica um estado de saúde da planta (ex: se está saudável, com sede ou encharcada) e envia para o Banco de Dados.
3. **Camada de Persistência (Banco de Dados):** Gerenciada pelo Firebase. Esta camada armazena de forma estruturada todo o histórico de leituras dos sensores, os diagnósticos gerados pela Inteligência Artificial e as informações de cadastro dos usuários, servindo como a fonte central de dados do projeto.
4. **Camada de Visualização (Frontend Mobile):** Desenvolvida em Flutter. É a interface gráfica (aplicativo mobile) com a qual o usuário interage. Ela consome os dados armazenados no Banco de Dados e as respostas em tempo real da API para exibir gráficos, alertas de saúde da planta e painéis de controle de forma visual e intuitiva.

## 4.2 Diagrama de Dados

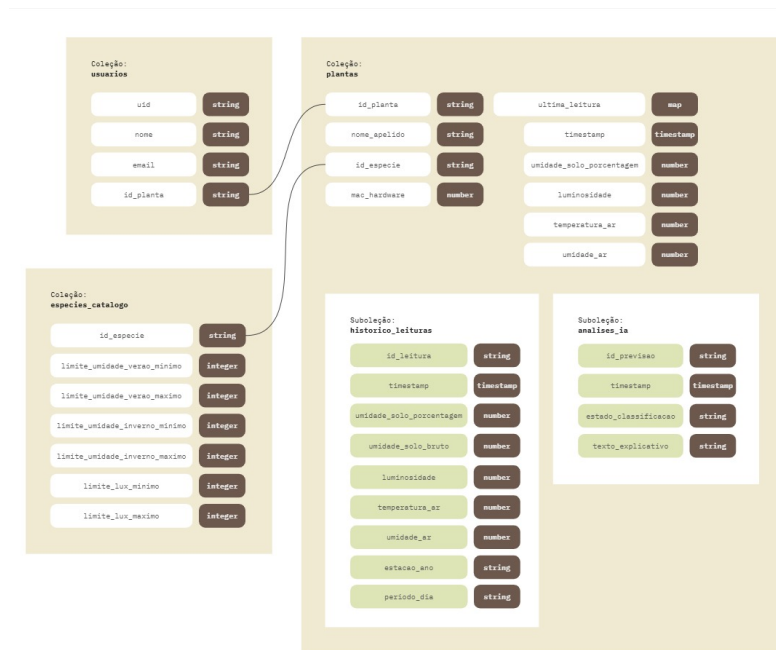


Figura 1 – Tabelas de dados.

### 4.3 Fluxo de Dados

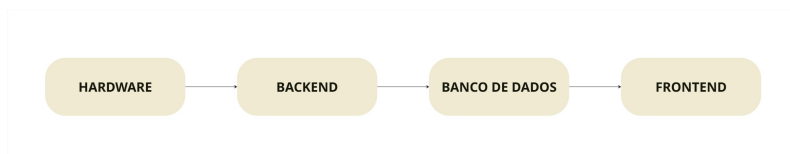


Figura 2 – Fluxo de dados geral.

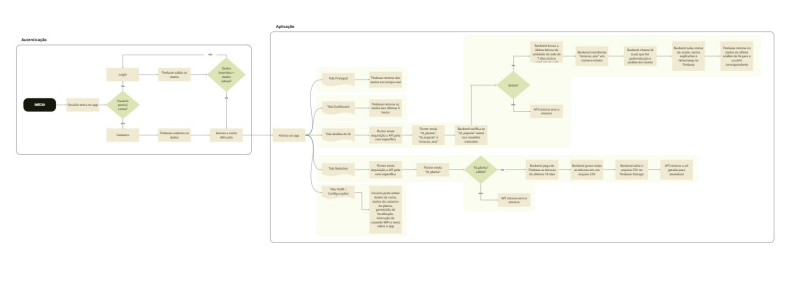


Figura 3 – Fluxo de dados específico.

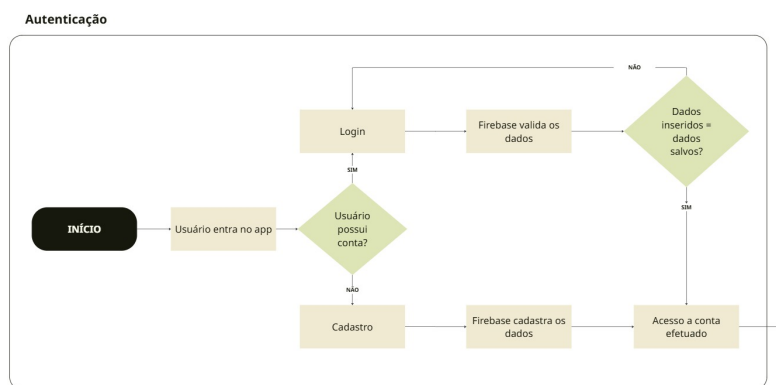


Figura 4 – Fluxo de dados da autenticação.



## 5.2 Esquema Elétrico e Montagem do Circuito

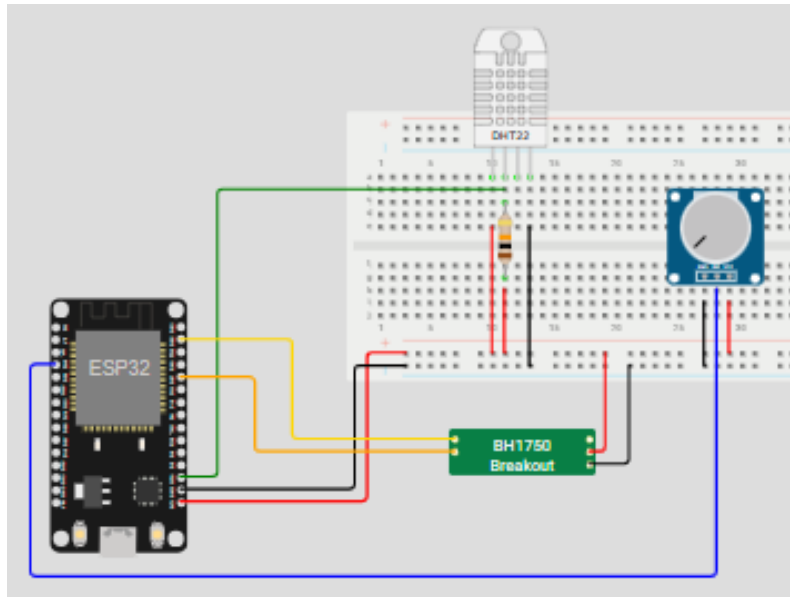


Figura 6 – Esquema elétrico.

## 5.3 Integração por API

A integração do sistema ocorre via protocolo HTTP, no qual o ESP32 realiza a coleta, processamento e envio de dados em quatro etapas principais:

1. **Normalização de dados:** O conversor ADC de 12 bits (0 a 4095) é mapeado para uma escala de 10 bits (0 a 1023), compatível com a camada de inteligência artificial. A umidade do solo também é convertida para uma escala percentual de 0% (seco) a 100% (encharcado), utilizando interpolação matemática com limitação por *constrain*.
2. **Identificação do dispositivo:** O endereço MAC do ESP32 é utilizado como identificador único do dispositivo e chave de autenticação no sistema.
3. **Estruturação dos dados:** As leituras de umidade do solo, luminosidade (lux), temperatura (graus celcius) e umidade do ar (%) são agrupadas com o MAC em um objeto JSON, posteriormente serializado para envio em rede.
4. **Envio dos dados:** O payload JSON é transmitido via requisição POST para o endpoint `/hardware/coleta` de uma API desenvolvida em FastAPI.

Na camada de backend, a API valida os dados utilizando *Pydantic* e associa as leituras ao dispositivo correspondente na coleção `plantas` do Firebase Firestore, utilizando o MAC como chave de busca.

Após a validação, são realizadas duas operações principais no banco de dados:



- Registro da leitura completa com timestamp (UTC) em uma subcoleção histórica do dispositivo;
- Atualização do documento principal com a última leitura recebida.

Essa atualização garante a sincronização em tempo real da interface do usuário desenvolvida em *Flutter*.

## 6 Inteligência Artificial (Modelo KNN)

### 6.1 O Algoritmo KNN

O algoritmo *K-Nearest Neighbors* (K-Vizinhos Mais Próximos, ou KNN) foi escolhido para este projeto por ser um modelo de aprendizado supervisionado simples e altamente eficiente para problemas de classificação. Como o objetivo do sistema é categorizar o estado de saúde da planta em classes bem definidas (como "Crítico", "Razoável" ou "Excelente"), o KNN se destaca por não exigir um treinamento matemático complexo. Ele funciona comparando os dados atuais recebidos pelos sensores com os dados históricos guardados em sua memória, classificando a planta com base nas características dos vizinhos mais próximos.

### 6.2 Geração de Dados Sintéticos

Para treinar o modelo de Inteligência Artificial, foi necessária a criação de um banco de dados inicial. Como o projeto ainda não possuía semanas de leituras reais acumuladas, foi realizado um processo de geração de dados sintéticos (dados simulados artificialmente via código). Esses dados foram gerados com base em estudos botânicos reais, simulando cenários ideais de umidade e situações de estresse hídrico para as plantas. Esse processo garantiu uma base de dados volumosa e equilibrada, essencial para que a IA aprendesse a diferenciar corretamente cada estado de saúde antes de ir para o ambiente real.

### 6.3 Variáveis Ambientais Consideradas

O modelo de Inteligência Artificial não toma decisões baseada apenas em um número isolado. Para garantir uma análise mais precisa, o algoritmo cruza duas variáveis ambientais fundamentais extraídas do histórico de monitoramento:

- **Umidade Atual do Solo (Bruta):** A leitura instantânea feita pelo sensor de umidade no momento exato da requisição. Ela indica o estado presente da terra.
- **Média de Umidade dos Últimos 7 Dias:** O histórico acumulado de umidade daquela planta na última semana. Essa variável é crucial porque permite que a IA entenda a tendência do solo, diferenciando uma rega recente de um solo que está constantemente encharcado ou seco.

## 6.4 Treinamento e Avaliação do Modelo

O desenvolvimento do modelo (apelidado carinhosamente de "Lee") foi realizado em ambiente Python. O banco de dados sintético foi dividido em duas partes: uma para o treinamento do algoritmo (80% dos dados) e outra menor para a realização de testes (20% dos dados). Durante a etapa de avaliação, o modelo apresentou excelentes métricas de desempenho (como altos índices de acurácia e precisão), provando ser capaz de classificar os estados de saúde sem confusões entre as classes. Após a validação do sucesso dos testes, o modelo treinado foi exportado e salvo em um arquivo de formato *.pkl* (Pickle), permitindo que ele seja facilmente carregado e utilizado pelo Backend a qualquer momento.

## 6.5 Estrutura Condicional de Status

O modelo KNN (Lee) trabalha classificando os dados em rótulos previsíveis, mas esses dados brutos não são amigáveis para o usuário final do aplicativo. Por isso, a API possui uma camada interna de tratamento de dados que funciona como um "tradutor" de estados.

Logo após o modelo KNN definir a classificação da planta, o Backend executa uma estrutura condicional (blocos *if/else*) para associar o resultado técnico a uma resposta humanizada. Essa estrutura define duas saídas principais:

- **Estado Classificado (Status):** A simplificação do diagnóstico em palavras-chave que alteram a interface do aplicativo, como "Excelente", "Crítico" ou "Razoável".
- **Texto Explicativo:** Uma mensagem personalizada e amigável que explica ao usuário se os cuidados estão atendendo à planta ou se há necessidade de atenção maior e replanejamento de cuidados.

## 6.6 Fluxo de Dados

A execução da Inteligência Artificial dentro do ecossistema do projeto segue um fluxo integrado e automatizado entre o aplicativo, o Backend e o banco de dados. O ciclo funciona da seguinte forma:

1. **Solicitação:** O usuário entra no aplicativo e solicita a análise de saúde da sua planta.
2. **Identificação:** O Backend recebe a requisição contendo o identificador da espécie (*id\_especie*).
3. **Busca de Dados:** O Backend conecta-se ao Firebase e busca as informações em tempo real de *umidade\_solo\_bruto* e *umidade\_ha\_7\_dias* associadas à conta do usuário (*uid*) e àquela planta específica.
4. **Processamento da IA:** O Backend aciona o modelo Lee (*.pkl*), enviando os dados recuperados do banco para que o algoritmo KNN faça a classificação do estado.

5. **Aplicação da Lógica:** Com o resultado da classificação da IA em mãos, o Backend aplica uma estrutura condicional para selecionar a mensagem de texto explicativa correspondente àquela situação (ex: um alerta amigável de socorro ou um parabéns pelo cuidado).
6. **Retorno:** O Backend devolve uma resposta contendo o status de saúde e o texto personalizado para o aplicativo do usuário correspondente.

## 7 Engenharia de Software e Backend

### 7.1 Estruturação com FastAPI

O microsserviço de backend do projeto *Lumees Yaplant* foi desenvolvido utilizando o *framework* assíncrono **FastAPI**, escolhido por sua alta performance, validação nativa de dados baseada em tipos complexos e geração automática de documentação. A arquitetura foi desenhada seguindo o princípio da modularidade, segregando as responsabilidades de negócio em roteadores específicos (*Routers*) para garantir a escalabilidade e a manutenibilidade do código-fonte.

### 7.2 Documentação dos Endpoints (Rotas da API)

A comunicação com o backend é estruturada sob o prefixo de versão `/lumees-api/v1`. A validação de contratos e a tipagem de entrada dos dados de requisições utilizam classes de dados estruturadas através da biblioteca **Pydantic**.

As rotas disponíveis na aplicação são:

- **POST `/lumees-api/v1/ia/analise`:** Recebe o ID da planta, a espécie e a estação do ano atuais. Processa os dados no modelo de inteligência artificial correspondente e retorna a classificação de saúde da planta.
- **POST `/lumees-api/v1/hardware/coleta`:** Ponto de entrada para o dispositivo ESP32. Recebe o endereço MAC do hardware e as leituras dos sensores (umidade do solo, luminosidade, temperatura e umidade do ar).
- **GET `/lumees-api/v1/plantas/{id_planta}/exportar-csv`:** Recebe o identificador de uma planta na URL para gerar o histórico de leituras em formato de planilha.

### 7.3 Integração Backend-Firebase

O backend integra-se com os serviços do Firebase por meio do SDK oficial (*firebase\_admin*), operando em duas frentes: banco de dados NoSQL (*Cloud Firestore*) e armazenamento de arquivos (*Cloud Storage*).

No **Cloud Firestore**, as operações realizam o gerenciamento dos dados do aplicativo:

- **Vínculo de Hardware:** A rota de coleta busca no banco qual planta possui o `mac_hardware` enviado pelo ESP32 para descobrir o `id_planta` correto.
- **Histórico de Sensores:** Cada nova leitura do hardware é salva na subcoleção `historico_leituras` com um `timestamp` em formato UTC. O campo `ultima_leitura` na raiz do documento também é atualizado para atualizar a interface do usuário.
- **Busca Retroativa:** Para executar a IA, o backend faz uma busca no histórico da planta para recuperar o registro de umidade do solo de exatos 7 dias atrás. Se não houver dados, o sistema adota a umidade atual como padrão de segurança.
- **Resultados da IA:** O diagnóstico gerado pelo modelo é armazenado no histórico de análises da planta e atualiza o status em tempo real no documento principal.

No **Cloud Storage**, o backend gerencia o fluxo de geração e exportação de relatórios da seguinte forma:

1. Busca no Firestore as leituras da planta referentes aos últimos 15 dias corridos.
2. Formata esses dados no padrão CSV (separados por ponto e vírgula) utilizando buffers de texto diretamente na memória RAM (`io.StringIO`), evitando gravação de arquivos temporários no servidor.
3. Envia o arquivo de texto gerado para uma pasta no Cloud Storage sob o caminho `relatorios/historico_{id_planta}.csv`.
4. Cria uma URL assinada (*Signed URL v4*) temporária e segura, com validade restrita de 1 hora e método `GET`, devolvendo esse link para o aplicativo realizar o download do arquivo.

## 8 Interface do Usuário

### 8.1 Fluxo de Navegação e Usabilidade (UX/UI)

O fluxo de navegação do aplicativo foi projetado para ser intuitivo, garantindo que o usuário acesse as informações de saúde da sua planta com o menor número de cliques possível. A jornada dentro do ecossistema mobile foi estruturada nas seguintes etapas sequenciais:

1. **Boas-Vindas:** O usuário interage inicialmente com a Tela Inicial, que apresenta uma mensagem de boas-vindas e o botão "Começar".
2. **Autenticação:** Ao avançar, o sistema direciona o usuário para a tela de autenticação, permitindo a escolha entre Criar Conta (para o primeiro acesso) ou o Login (para quem já possui uma conta ativa).

3. **Tela Principal Dinâmica:** Após a validação do acesso, o usuário entra na Tela Principal. Esta tela possui uma lógica condicional inteligente baseada no banco de dados:
  - *Sem planta cadastrada:* A interface esconde o avatar da planta e exibe um cartão (*card*) de destaque solicitando o cadastro do primeiro dispositivo e da espécie.
  - *Com planta cadastrada:* O cartão de aviso desaparece, dando lugar ao avatar gráfico da planta, que muda de expressão de acordo com o estado de saúde atual.
4. **Configuração de Rede (Wi-Fi):** Para que os dados comecem a ser transmitidos, o usuário precisa conectar o hardware à rede local. Caso seja a primeira vez, o aplicativo oferece instruções claras na tela de configurações para realizar esse vínculo; se a rede já estiver configurada, o sistema inicia a comunicação automaticamente.

Uma vez estabelecida a conexão e o sensor inserido no solo, o usuário ganha acesso livre para navegar pelas cinco telas principais de gerenciamento do aplicativo:

- **Monitoramento em Tempo Real (Tela Principal):** Exibe os dados instantâneos enviados pelos sensores e as reações visuais da planta.
- **Análise Histórica (Tela Dashboard):** Apresenta gráficos contendo as leituras coletadas nas últimas 5 horas, permitindo acompanhar a evolução recente do ambiente.
- **Diagnóstico Avançado (Tela Diagnóstico IA):** Espaço onde o usuário pode solicitar manualmente uma análise aprofundada para o modelo de Inteligência Artificial (Lee).
- **Exportação de Dados (Tela Relatório):** Permite gerar e baixar um arquivo em formato CSV com o histórico completo de medições dos últimos 15 dias.
- **Gerenciamento do Perfil (Tela de Usuário):** Pannel onde é possível visualizar e editar dados pessoais (nome, e-mail e senha), gerenciar permissões de privacidade do sistema (como o uso da localização) e consultar o manual de instruções de configuração do Wi-Fi.

## 8.2 Identidade Visual e Elementos Gráficos

## 8.3 Detalhamento das Telas Desenvolvidas

### 8.3.1 Telas de Acesso (Inicial, Login e Cadastro)

A jornada se inicia na **Tela Inicial**, uma interface limpa com foco em conversão que apresenta a marca do projeto e direciona o usuário ao fluxo de entrada. A **Tela de Login** e a **Tela de Cadastro** possuem campos validados para inserção de credenciais (e-mail e senha). O cadastro armazena as informações básicas no Firebase para criar o perfil único do usuário (*uid*).

### 8.3.2 Tela Principal (Monitoramento Real-Time)

Esta é a tela de maior permanência do usuário. Ela consome dados diretamente do banco de dados para renderizar seu conteúdo de forma dinâmica. Caso o sistema identifique que o usuário não possui um dispositivo atrelado, exibe-se um cartão de ação (*card*) centralizado para o cadastro. Havendo um dispositivo ativo, a tela exibe os dados de leitura instantânea dos sensores e o avatar gráfico da planta, cujo estado visual (expressão e animação) responde diretamente aos dados de saúde calculados pelo Backend.

### 8.3.3 Tela Dashboard (Gráficos Históricos)

Voltada para a análise de tendências, esta tela organiza as leituras brutas capturadas pelos sensores em uma linha do tempo retroativa de até 5 horas. Os dados são plotados em gráficos de linha dinâmicos, permitindo que o usuário visualize de forma simples se a umidade e a temperatura do ambiente estão se mantendo estáveis ou se apresentam picos de anomalia.

### 8.3.4 Tela de Diagnóstico da Inteligência Artificial

Esta interface funciona como um painel de consulta avançada. Nela, o usuário encontra um botão de ação rápida para disparar uma requisição de análise. Ao ser acionada, a tela exibe o veredito do modelo KNN (Lee) de forma destacada (ex: "Crítico", "Saudável") acompanhado do bloco de texto explicativo gerado pela API.

### 8.3.5 Tela de Relatório (Exportação CSV)

Uma tela utilitária projetada para usuários que necessitam de um registros mais profundos. Ela exibe um botão para exportar e baixar uma tabela estruturada em formato CSV contendo todas as medições coletadas nos últimos 15 dias, facilitando o compartilhamento e a leitura externa dos dados.

### 8.3.6 Tela de Configurações e Usuário

Interface administrativa dividida em três blocos principais de controle:

- **Dados do Perfil:** Permite a edição segura de nome, e-mail e redefinição de senha.
- **Privacidade e Permissões:** Chaves de ativação de recursos do sistema, como a permissão para uso da localização do dispositivo mobile.
- **Suporte Técnico (Wi-Fi):** Mini guia com instruções para que o usuário saiba como parear o hardware do sensor à rede sem fio local.

## Conclusão e Trabalhos Futuros

O Lumees eleva o nível do cuidado com plantas ao unir IoT, computação em nuvem e inteligência artificial em uma única solução inteligente. Mais do que monitorar

dados, o sistema elimina as incertezas do cultivo, oferecendo acompanhamento preciso, interativo e acessível. Com potencial de expansão para novas espécies e automação completa da rega, o Lumees se posiciona como a plataforma definitiva para transformar a forma como as pessoas cuidam de suas plantas.

#### 8.4 Propostas de Melhorias Futuras

- IA especializada para diferentes espécies de plantas.
- Rega autônoma baseada em decisões inteligentes.
- Plataforma escalável preparada para o futuro da jardinagem conectada.
- Expansão da Inteligência Artificial Lee para novas espécies e perfis de cultivo.
- Evolução do avatar virtual, tornando a interação mais dinâmica e personalizada.
- Experiência gamificada para aumentar o engajamento do usuário.
- Criação de recomendações automáticas de cuidados para plantas.
- Ampliação do histórico analítico complexo com previsões e tendências de saúde vegetal.
- Consolidação do Lumees como uma plataforma completa para jardinagem inteligente.

## Referências